Project Errors Report

Network Digital Twin

Next.js 16 + TypeScript + Prisma 7 + LibSQL

2026-04-07

Total: 15 problems found (4 critical, 9 fixed, 2 remaining)

1. Critical Errors (blocked launch)

These errors completely prevented the project from starting. They were caused by misconfiguration of the database connection, Prisma client generation, and stale artifacts from a previous project located in the parent directory. Each of these had to be resolved before any code could compile or run.

1.1 Missing .env file

Prisma could not find the DATABASE_URL environment variable. The prisma.config.ts file reads process.env["DATABASE_URL"], but no .env file existed in the project root. Additionally, the lib/prisma.ts fallback path pointed to a database file that might not exist.

```
Error: DATABASE_URL is not set in prisma.config.ts
```

Fix: Created .env file with DATABASE_URL pointing to the SQLite database.

```
DATABASE_URL="file:/home/z/my-project/db/custom.db"
```

1.2 Conflicting Prisma clients from parent project

The parent directory /home/z/my-project/node_modules/.prisma/client contained a Prisma client generated from a completely different schema. This old schema had fields like current_nom, connections_from, impedance_z, and from_id, while the current project uses elementId, parentId, voltageLevel, and sourceId. TypeScript resolved to the old client, causing type mismatches on every Prisma query. This was the most insidious error because it manifested as "field does not exist" type errors rather than an obvious module conflict.

```
Type error: 'elementId' does not exist in type 'ElementSelect'
(actual fields: id, type, name, description, parent_id, location, ...)
```

Fix: Removed the stale Prisma client from the parent directory.

```
rm -rf /home/z/my-project/node_modules/.prisma
```

1.3 Prisma 7 generator missing output path

Prisma 7 with the "prisma-client" generator requires an explicit output directory to be specified in the schema file. The schema only had the provider declared, causing the generate command to fail with a clear error message. Previous versions of Prisma did not require this.

```
Error: An output path is required for the prisma-client generator.
Please provide an output path in your schema file.
```

Fix: Added output path to prisma/schema.prisma generator block.

```
generator client {
  provider = "prisma-client"
  output   = "../node_modules/.prisma/client"
}
```

1.4 Database schema drift

The SQLite database file (custom.db) contained tables and columns from a completely different schema version with 25+ tables, additional indexes, and different column names. Prisma migrate detected the drift and refused to apply migrations, requiring a full reset. This happened because the same database file was shared between two separate projects at different times.

```
Error: Drift detected: Your database schema is not in sync
with your migration history.
We need to reset the SQLite database.
```

Fix: Forced database reset and schema push.

```
npx prisma db push --force-reset
```

2. TypeScript Errors (implicit any from Prisma 7)

Prisma 7 with the prisma-client provider generates TypeScript source files (.ts) instead of compiled JavaScript. While Next.js Turbopack can compile these, the @prisma/client runtime package does not provide type information for query results at build time. This means every Prisma query returns a value typed as "any", and with strict: true in tsconfig.json, any callback parameter (map, filter, reduce) on these results triggers an implicit any error. A total of 17 type errors were found across 3 files, all requiring manual type annotations.

2.1 app/api/network/route.ts - 4 type errors

The network API endpoint builds an element lookup map and enriches connection data with source and target element information. All three operations involved untyped callback parameters on Prisma query results: the .map() call for building the Map, and the .map() call for enriching connections. The Map constructor and all property accesses on elements failed type checking.

```
Error: Parameter 'e' implicitly has an 'any' type. (line 28)
Error: Property 'elementId' does not exist on type '{}'. (line 39)
```

Fix: Added explicit type aliases and annotations.

```
type ElementRow = { id: string; elementId: string; name: string;
                  type: string; posX: number | null; posY: number | null };
type ConnRow = { id: string; sourceId: string; targetId: string };
const elementMap = new Map&lt;string, ElementRow&gt;(
  elements.map((e: ElementRow) =&gt; [e.id, e])
);
connections.map((conn: ConnRow) =&gt; {
  const src = elementMap.get(conn.sourceId);
  // ...
});
```

2.2 app/api/stats/route.ts - 3 type errors

The stats endpoint uses .filter() on Prisma element results and .reduce() on load/transformer results. Both operations had callback parameters typed as implicit any. The reduce accumulator also lacked type annotation, causing the return type inference to fail.

```
Error: Parameter 'e' implicitly has an 'any' type. (line 12)
Error: Parameter 'l' implicitly has an 'any' type. (line 24)
Error: Parameter 't' implicitly has an 'any' type. (line 27)
```

Fix: Added inline type annotations to all callbacks.

```
elements.filter((e: { type: string }) =&gt; toLower(e.type) === 'source')
loadDevices.reduce((sum: number, l: { powerP: number }) =&gt; sum + l.powerP, 0)
transformers.reduce((sum: number, t: { power: number }) =&gt; sum + t.power, 0)
```

2.3 app/api/validation/route.ts - 8 type errors

The validation endpoint has the most complex data access patterns, using deeply nested Prisma includes (element -> deviceSlots -> devices -> breaker/load, and element -> sourceConnections -> cable). Each level of nesting creates callback parameters that need type annotations. Specifically, flatMap on deviceSlots, filter/map on devices, and filter/map on sourceConnections all required explicit types.

```
Error: Parameter 's' implicitly has an 'any' type. (line 69)
Error: Parameter 'd' implicitly has an 'any' type. (line 70, 103)
Error: Parameter 'c' implicitly has an 'any' type. (line 73)
```

Fix: Added type annotations to all nested callbacks.

```
element.deviceSlots
  .flatMap((s: { devices: any[] }) =&gt; s.devices)
```

```
  .filter((d: { breaker: any }) =&gt; d.breaker)
  .map((d: { breaker: any }) =&gt; d.breaker!)

element.sourceConnections
  .filter((c: { cable: any }) =&gt; c.cable)
  .map((c: { cable: any }) =&gt; c.cable!)
```

2.4 lib/utils/id-generator.ts - 2 type errors

The ElementType type union includes "JUNCTION", but two Record<ElementType, string> objects in generateElementId() and generateElementName() functions did not include a JUNCTION key. TypeScript correctly reports this as a type error because Record requires all keys of the union type to be present.

```
Error: Type 'string' is not assignable to type 'never'.
(JUNCTION key missing from Record)
```

Fix: Added JUNCTION entries to both Record objects.

```
const prefixes: Record&lt;ElementType, string&gt; = {
  SOURCE: 'SRC', CABINET: 'CAB', LOAD: 'LOAD', BUS: 'BUS',
  METER: 'MTR', BREAKER: 'QF', JUNCTION: 'JNC',  // added
};
```

3. Import Path Errors

Two utility scripts referenced an outdated Prisma client import path that was used in an earlier version of the project setup. The path pointed to a generated directory that no longer exists after switching to the standard @prisma/client import.

3.1 prisma/seed.ts - outdated import path

The database seed script imported PrismaClient from a non-standard generated path. This path was created during an earlier project iteration when Prisma was configured with a different output directory. After standardizing the setup, the import path became invalid.

Fix: Changed import to standard @prisma/client package.

```
// Before:
import { PrismaClient } from '../app/generated/prisma/client';
// After:
import { PrismaClient } from '@prisma/client';
```

3.2 scripts/import-data.ts - outdated import path

Same issue as the seed script. The import-data script used the same non-standard generated path for PrismaClient, causing a module resolution failure at runtime.

Fix: Same correction as seed.ts.

4. Remaining Issues (not fixed)

These issues were identified during the audit but deliberately not fixed, either because they require significant refactoring (validation service rewrite) or because they are minor UI inconsistencies that do not block functionality.

4.1 Validation voltage drop calculation is a stub

In app/api/validation/route.ts lines 111-112, the voltage drop check uses a hardcoded value of 2% instead of performing an actual calculation. The real calculation functions exist in lib/calculations/voltageDrop.ts but are not connected to the validation route. This means the voltage drop check will always pass (2 < 4) and never report issues, even when actual voltage drops exceed the 4% limit. The proper fix would require integrating the voltage drop calculation with the path-tracing logic from the network graph.

```
// Line 111-112 in validation/route.ts:
const voltageDrop = 2; // STUB - always 2%
if (voltageDrop > 4) { ... } // never triggers
```

4.2 validation.service.ts not adapted for new schema

The old project archive contains a comprehensive validation service (src/lib/services/validation.service.ts, 536 lines) with 4 rules: CABLE_001 (breaker-cable coordination), VOLTAGE_001 (voltage drop), PROT_001 (protection sensitivity), and SEL_001 (selectivity). However, this service uses the old database schema with snake_case fields (current_nom, connections_from, from_id, impedance_z, etc.) and a flat Element-Device model instead of the current Element-DeviceSlot-Device-Breaker hierarchy. Adapting it requires rewriting all database queries, path-finding algorithms, and field access patterns. The current validation is done inline in the API route with simplified logic.

4.3 Hardcoded import file path

In lib/services/import.service.ts line 19, the Excel file path is hardcoded to a specific upload file. There is no mechanism to upload a different file through the UI or pass a file path as a parameter to the import API endpoint.

```
const INPUT_FILE_PATH = '/home/z/my-project/upload/...xlsx';
```

4.4 Cabinets not shown in stats panel

The API /api/stats returns a cabinets count, and the TypeScript interface Stats includes the cabinets field. However, the statistics panel in page.tsx only displays 6 categories: sources, buses, breakers, meters, loads, and junctions. The cabinets category is missing from the grid layout, even though cabinets are one of the most important element types for the electrical network hierarchy visualization.

4.5 Two projects in shared filesystem

Both /home/z/my-project/ (old project with src/ directory) and /home/z/my-project/network-digital-twin/ (current project) exist in the same parent directory. This caused the Prisma client conflict (Error 1.2) and can cause confusion with node_modules resolution, TypeScript path aliases, and environment configuration. The old project directory should ideally be removed or relocated to prevent future conflicts.

5. Summary Table

#	Severity	File	Description	Status
1	CRITICAL	.env	Missing DATABASE_URL	FIXED
2	CRITICAL	node_modules/.prisma/	Conflicting Prisma client from parent project	FIXED
3	CRITICAL	prisma/schema.prisma	Generator missing output path	FIXED
4	CRITICAL	custom.db	Database schema drift	FIXED
5	HIGH	app/api/network/route.ts	4 implicit any type errors	FIXED
6	HIGH	app/api/stats/route.ts	3 implicit any type errors	FIXED
7	HIGH	app/api/validation/route.ts	8 implicit any type errors	FIXED

8	MEDIUM	lib/utils/id-generator.ts	Missing JUNCTION in Record	FIXED
9	MEDIUM	prisma/seed.ts	Outdated PrismaClient import path	FIXED
10	MEDIUM	scripts/import-data.ts	Outdated PrismaClient import path	FIXED
11	MEDIUM	app/api/validation/route.ts	Voltage drop hardcoded stub (2%)	REMAINING
12	LOW	lib/services/validation.service.ts	Not adapted for new schema	REMAINING
13	LOW	lib/services/import.service.ts	Hardcoded Excel file path	REMAINING
14	LOW	app/page.tsx	Cabinets missing from stats panel	REMAINING
15	INFO	/home/z/my-project/	Two projects in shared filesystem	REMAINING